Design and Verification of AMBA AXI-LITE Protocol

ECE 571 – INTRODUCTION TO SYSTEM VERILOG

Under the Guidance of
Prof. Jean Paul Mugisha

TEAM MEMBERS
Nikhil Swarna
Venkat Sai Sumanth Koyada
Hanisha Dhananjaya Produtur
Sajida Sayyad

I. INTRODUCTION

Advanced eXtensible Interface (AXI) is a high- bandwidth communication protocol used for on-chip communication between the processor, memory, and other components. AXI is commonly used in system-on-a-chip (SoC) and FPGA. The AXI protocol, introduced in 2003 as part of the AMBA (Advanced Microcontroller Bus Architecture) standard developed by ARM. The three types of AXI interface are AXI (full), AXI-lite, and AXI-stream. Axi full is commonly used in memory-mapped environments where high bandwidth data transfer is required. AXI-lite is suitable for lower throughput memory-mapped communication, such as control and status registers. AXI-stream is for high-speed streaming of data (one way) . One of the most used protocols for on-chip buses in modern Wireless Sensor Network SoCs is AMBA AXI4. A SoC design's interconnectivity is just as vital as the parts or blocks it incorporates. For high-speed SoC interconnects, the AMBA AXI4 is seen as an appropriate interface. It makes it easier for processors and hardware accelerators (HAs) in WSNs to communicate with one another. Additionally, it minimizes size and power consumption while supporting high-performance on-chip connectivity.

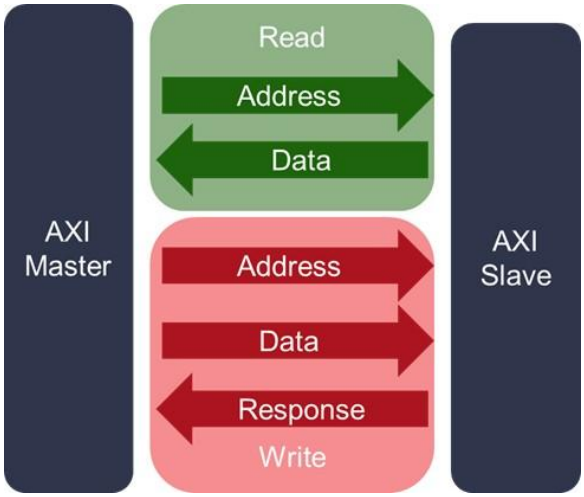

Fig-1. AMBA AXI Lite Read Write Architecture

Global	ACLK	ARESETn	—	—
Write address channel	AWVALID	AWREADY	AWADDR	AWPROT
Write data channel	WVALID	WREADY	DATA	WSTRB
Write response channel	BVALID	BREADY	BRESP	—
Read address channel	ARVALID	ARREADY	ARADDR	ARPROT
Read data channel	RVALID	RREADY	RDATA	RRESP

Table-1 Global Signals and Local Signals in each channel

As we can see that there are about five transaction channels which are the means of communication between Master and Slave as shown in the Figure. There is one AXI Master and the other is the AXI Slave, in between there an AXI Interconnect. The Five transaction channels are as follows:

1. *Write Address Channel*
2. *Write Data Channel*
3. *Write Response Channel*
4. *Read Address Channel*
5. *Read Data Channel*

II.OVERVIEW

The Advanced Microcontroller Bus Architecture (AMBA), developed by ARM, is a widely adopted on-chip communication standard used to connect processors, memory, and peripherals in System-on-Chip (SoC) designs. As a lightweight subset of the AXI4 standard, the AXI4-Lite protocol is specifically designed for simple, single-beat register-level transfers where burst transactions and exclusive accesses are not required. It communicates over five independent channels Write Address (AW), Write Data (W), Write Response (B), Read Address (AR), and Read Data (R) each governed by a VALID/READY handshake mechanism that ensures reliable data transfer only when both the master and slave are simultaneously ready.

This project implements a complete AXI4-Lite communication subsystem in System Verilog, comprising a protocol-compliant master interface and an 8-register memory-mapped slave device. The master drives both single and multi-beat read and write transactions using handshake-compliant signalling, while the slave processes incoming requests through independent FSM-controlled read and write data paths. The slave supports byte-lane selective writes using write strobe signals and enforces well-defined response rules — returning OKAY for valid transactions, SLVERR for writes to a protected read-only register, and DECERR for out-of-range or misaligned addresses.

The verification environment is built around a self-checking testbench that validates the design across four structured test scenarios, covering normal read/write operations, error response handling, and mixed transaction sequences that interleave all three AXI response types. Each transaction result is automatically compared against an expected value, with a cumulative pass/fail summary reported at the end of simulation. The project demonstrates the practical application of System Verilog constructs including interfaces with mod ports, packages, object-oriented BFM classes, and FSM-based RTL in building a modular, reusable, and fully verified protocol subsystem.

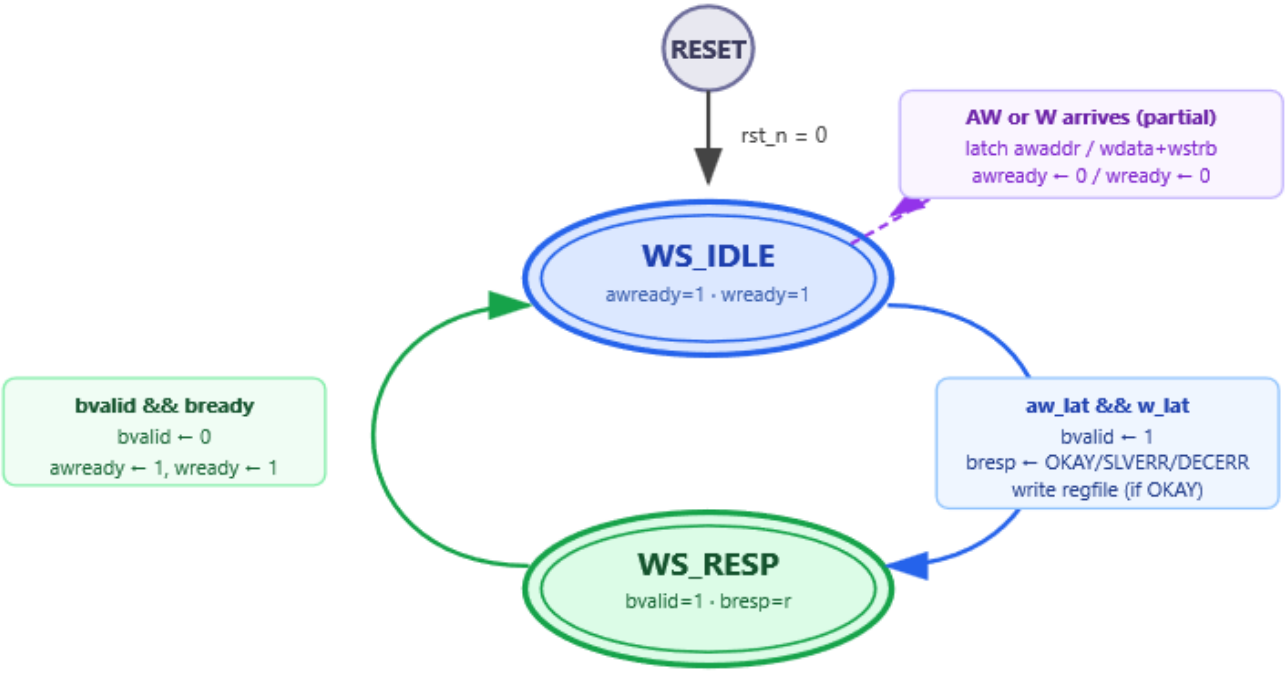

Fig-2. FSM for Write Operation.

The Write FSM operates with two states WS_IDLE and WS_RESP. In WS_IDLE, the slave independently latches the address (AW channel) and data (W channel), and once both are received, it evaluates the response type (RESP_OKAY, RESP_SLVERR, or RESP_DECERR), writes to the register file if valid, and transitions to WS_RESP with bvalid=1. In WS_RESP, it waits for the master to assert bready, then clears bvalid and returns to WS_IDLE with both ready signals reasserted.

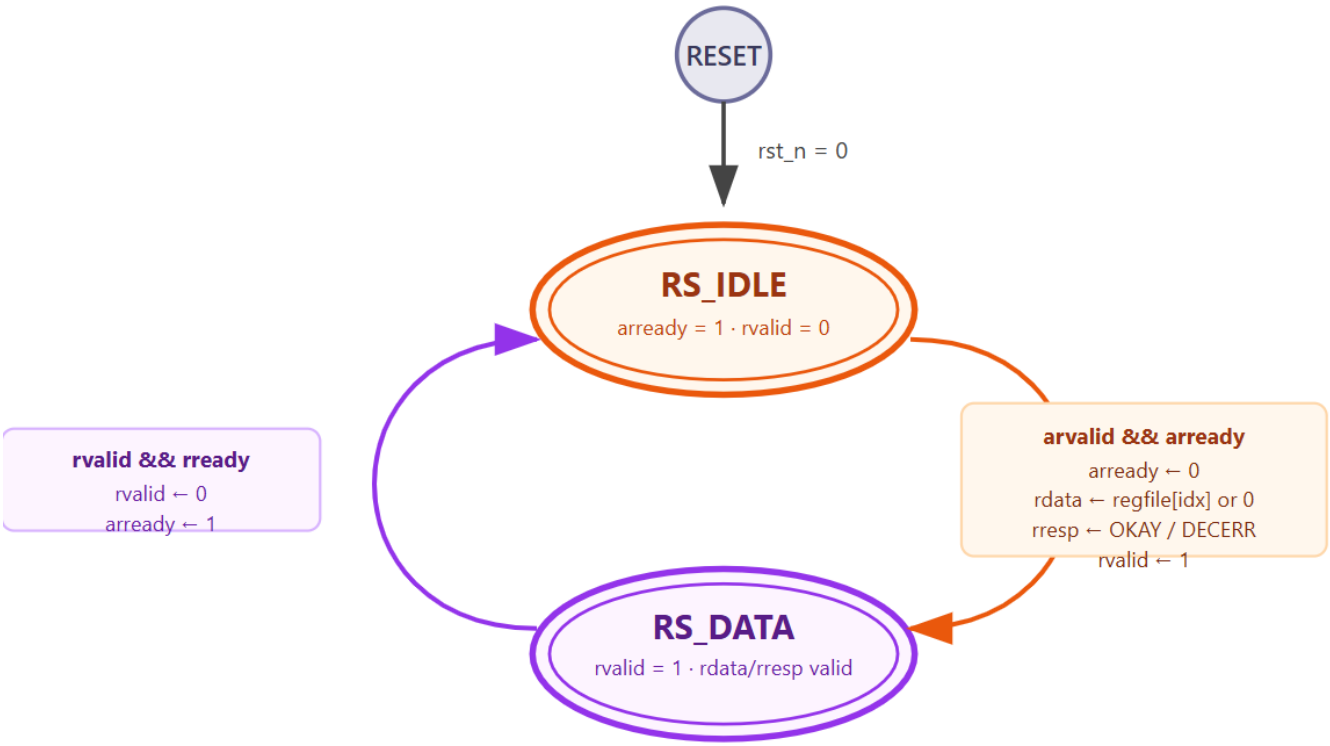

Fig-3. FSM for Read Operation.

The Read FSM operates with two states RS_IDLE and RS_DATA. In RS_IDLE, the slave waits for the master to assert arvalid, and upon a successful handshake it captures the read address, drives the appropriate rdata from the register file and sets rresp to either RESP_OKAY (valid aligned address) or RESP_DECERR (out-of-range or misaligned), then transitions to RS_DATA with rvalid=1. In RS_DATA, it waits for the master to assert ready, then clears rvalid and returns to RS_IDLE with arready reasserted, ready for the next read transaction.

III. VERIFICATION METHODOLOGY

To verify the AXI4-Lite slave design, a structured System Verilog testbench was developed using interfaces, tasks, and self-checking mechanisms. A reusable testbench interface was created to generate manager-side stimulus targeting the slave's 8-register file (reg [0]–reg [7]), where reg[0]–reg [6] are read-write (pre-loaded with 0xC0DE_000x) and reg [7] is hardwired read-only (0xDEAD_BEEF). The manager module translates these signals into AXI channel activity, ensuring all five protocol channels AW, W, B, AR, and R are exercised in simulation.

A key component of the verification strategy was the development of reusable read() and write() tasks that drive the AW/W/B and AR/R channels in the correct handshake order. Each transaction automatically checks BRESP and RRESP against the expected response RESP_OKAY for valid aligned writable addresses, RESP_SLVERR for write attempts to read-only reg[7], and RESP_DECERR for out-of-range (addr > 0x1C) or misaligned accesses with $fatal halting simulation immediately on any mismatch.

The verification followed a layered methodology covering unit-level, integration-level, and corner-case testing. Unit testing validated the write FSM (WS_IDLE ↔ WS_RESP) and read FSM (RS_IDLE ↔ RS_DATA) independently for correct RESET and handshake behavior. Integration testing then connected the manager and slave through the AXI interface and confirmed end-to-end register read-write correctness. Corner-case tests covered: aligned reads and writes to all eight registers, write attempts to read-only reg[7] expecting RESP_SLVERR, misaligned and out-of-range addresses expecting RESP_DECERR, boundary writes to the last valid address (0x1C), WSTRB byte-masking behaviour across

reg[0]–reg[6], read-after-write sequences to confirm data persistence, and AR-channel back-pressure stalling.

IV. CHALLENGES

An original handshake module was used to accept data on READY and VALID, but it did not drive ready and valid; this resulted in manually driving too many ready and valid lines. Therefore, an FSM approach was tried.

One of the main challenges for this is in getting the correct timing for the FSM. Most of the issues in building the FSM were logical errors, since all the signals and values at the current and next state needed to be kept in mind at the same time. Originally, the FSM included IDLE, SEND, RESET, and HOLD; this resulted in an extra cycle after receiving data where nothing happened. Many signals had to be moved around in the FSM since the HOLD state was not functional. Ultimately, what worked was to code in English at a high level (in the comments), then translate to Verilog. During testing, the tx_channel module failed due to a pass-through glitch, which resulted in values not holding during the hold phase. Changes that fixed issues in one testbench resulted in an error in another, and vice versa. This was resolved by adding a register in the module specifically to hold the data and changing this module synchronously. Identification of where to logically change data to prevent glitches in an FSM (synchronous always_ff block or always_comb block) is deemed to be difficult.

The team was split into design and verification. When someone from verification needs to test a module, they have to spend additional time understanding the module. This added time to the design, verification, design cycle. It was determined that it would be more efficient for designers to test and verify, and debug their own modules to some degree, before passing them on to the verification team. The verification team would connect the two modules designed by different designers and test the final functionality.

V. SIMULATION RESULTS

```
#   ============================================================
#   AXI-LITE RESPONSE CODE TESTBENCH
#   Covers: OKAY  |  SLVERR  |  DECERR  +  normal operations
#   reg[7] addr=0x1C is READ-ONLY  (write -> SLVERR)
#   ============================================================
#
#   SECTION 4   MIXED  â  OKAY + SLVERR + DECERR interleaved
#   Real-world mix: valid ops, protected reg writes, bad addresses
#
#   STEP   OP        ADDR        DATA        RESP     RESULT   NOTE
#   ------ --------- ----------  ----------  ------   ------   -------------------------
#   W1     WRITE     0x00000010  0xface0001  OKAY     PASS     exp=OKAY
#   W2     WRITE     0x000000ff  0xbad00006  DECERR   PASS     exp=DECERR
#   W3     WRITE     0x0000001c  0xbad00007  SLVERR   PASS     exp=SLVERR
#   R1     READ      0x00000010  0xface0001  OKAY     PASS     exp_data=0xface0001  exp_resp=OKAY
#   R2     READ      0x0000001c  0xdeadbeef  OKAY     PASS     exp_data=0xdeadbeef  exp_resp=OKAY
#   R3     READ      0x000000ff  0x00000000  DECERR   PASS     exp_data=0x00000000  exp_resp=DECERR
#   W4     WRITE     0x00000014  0xcafebabe  OKAY     PASS     exp=OKAY
#   R4     READ      0x00000014  0xcafebabe  OKAY     PASS     exp_data=0xcafebabe  exp_resp=OKAY
#   W5     WRITE     0x00000002  0xbad00008  DECERR   PASS     exp=DECERR
#   W6     WRITE     0x00000004  0xbeef1234  OKAY     PASS     exp=OKAY
#   R5     READ      0x00000004  0xbeef1234  OKAY     PASS     exp_data=0xbeef1234  exp_resp=OKAY
#   ------ --------- ----------  ----------  ------   ------   -------------------------
#
#   ============================================================
#   SUMMARY
#   ============================================================
#   Total checks   11
#   PASS           11
#   FAIL           0
#
#   ALL CHECKS PASSED
#   ============================================================
```

Fig-4. Transcript for the section 4 (Combination of all possible responses- OKAY, SLVERR, DECERR)

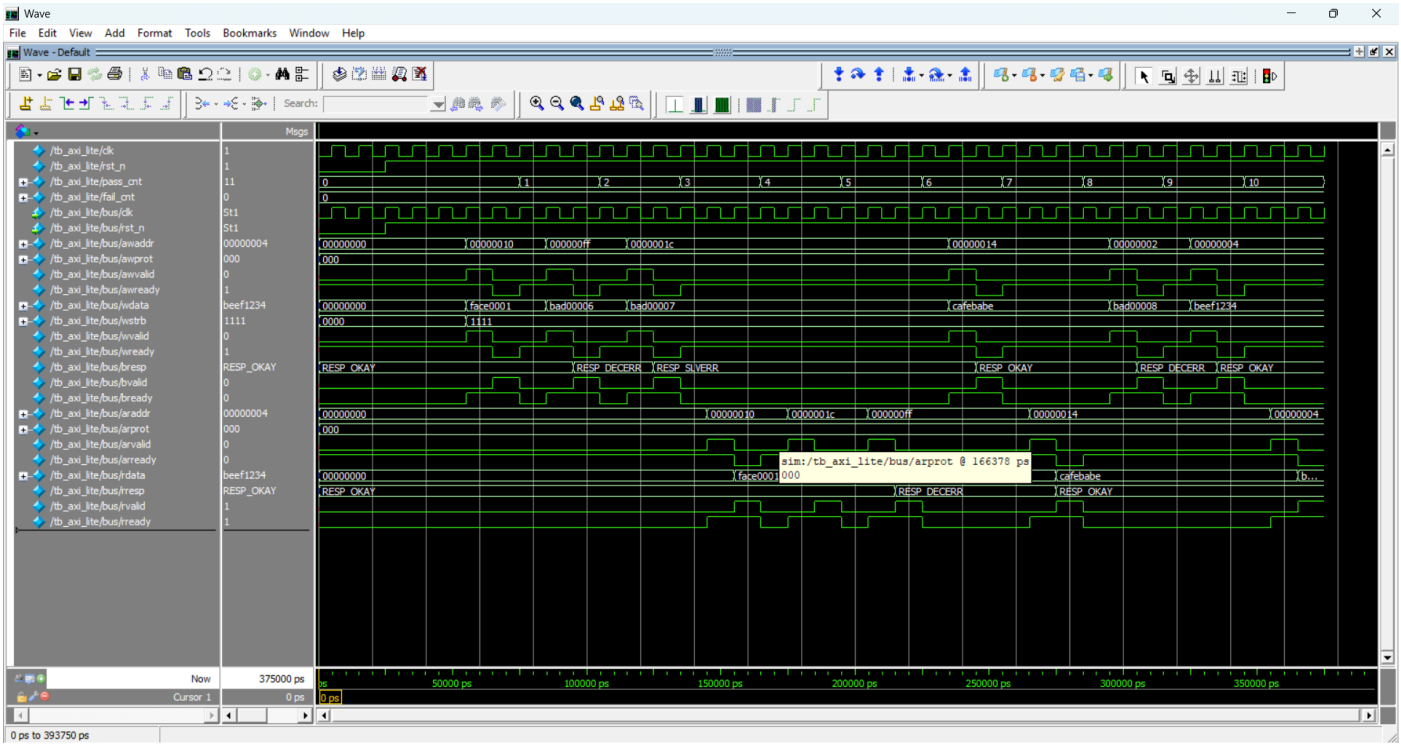

Fig-5. Waveform for the section 4 (Combination of all possible responses- OKAY, SLVERR, DECERR)

VI.ROLES AND RESPONSIBILITES

Sajida Sayyad: Interface & Master Write Channel

- Designs and maintains the AXI Lite interface, ensuring correct signal definitions and connectivity between master and slave.
- Implements write address and data channels by controlling AWVALID, WVALID, and handling handshake signals properly.
- Handles write response channel by receiving BRESP and completing transactions using BREADY.
- Debugs write-side issues and ensures protocol-compliant data transfer with coordination from the slave write logic.

Nikhil Swarna: Master Read Channel & Slave Read Path

- Implements read address and data channels by managing ARVALID, RREADY, and ensuring correct handshake behaviour.
- Develops slave-side read logic to decode addresses and return appropriate data through RDATA.
- Ensures proper sequencing between address and data phases to maintain correct read transactions.
- Debugs read path functionality and verifies correctness of returned data with testbench support.

Hanisha Dhananjaya Produtur: Slave Write Logic & Package

- Implements slave write channels by handling AWREADY, WREADY, and updating registers/memory correctly.
- Generates write responses using BVALID and BRESP to complete transactions reliably.
- Maintains the package file by defining parameters, typedefs, and ensuring consistency across modules.
- Verifies data integrity after writes and coordinates with master write logic for correctness.

Venkat Sai Sumanth Koyada: Testbench & Integration

- Developed a comprehensive testbench by integrating all modules, generating clock/reset, and ensuring correct system-level connectivity.
- Implemented reusable automatic tasks for read and write operations to modularize stimulus generation and improve verification efficiency.
- Designed and executed multiple test scenarios including OKAY, SLVERR, and DECERR responses, validating protocol behavior under normal and error conditions.
- Identified and debugged critical issues during simulation, resolving handshake mismatches and data inconsistencies through iterative fixes and analysis.

VII. REFERENCES

- https://www.arm.com/architecture/system-architectures/amba/amba-4
- https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/axi.html
- M. Makni, M. Baklouti, S. Niar and M. Abid, "Performance Exploration of AMBA AXI4 Bus Protocols for Wireless Sensor Networks," 2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA), Hammamet, Tunisia, 2017, pp. 1163-1169, doi: 10.1109/AICCSA.2017.26. keywords: {Protocols;System-on-chip;Field programmable gate arrays;Optimization;Benchmark testing;Wireless sensor networks;Computer architecture;SoC;HLS;AXI4;FPGA;AMBA;Hardware Accelerator},
- https://www.realdigital.org/doc/6cb4e88c145c960ad199be9fac5e9402
- AXI Basics 1 - Introduction to AXI
- https://deepwiki.com/pulp-platform/axi/2-axi-protocol-fundamentals
- https://developer.arm.com/community/arm-community-blogs/b/soc-design-and-simulation-blog/posts/introduction-to-axi-protocol-understanding-the-axi-interface
- https://www.scribd.com/document/347900414/amba3james0801-121106095203-phpapp01-151203125056-lva1-app6891